

perm:role:{roleName}

Example:

perm:role:AUDITOR

Value stored as SET:

AUDIT_CREATE
AUDIT_EDIT
WORKPAPER_UPLOAD

Redis Service Example:

```
public String getUserRole(String tenantId,String userId,String assessmentId){  
  
    String key = "role:user:" + tenantId + ":" + userId + ":" + assessmentId;  
  
    return redisTemplate.opsForValue().get(key);  
}
```

```
public Set<String> getPermissionsForRole(String role){  
  
    String key = "perm:role:" + role;  
  
    return redisTemplate.opsForSet().members(key);  
}
```

6. CACHE INSERTION WITH TTL

When cache miss occurs we fetch from DB and store in Redis.

```
public void cacheUserRole(String tenantId,String userId,String assessmentId,String role){  
  
    String key = "role:user:" + tenantId + ":" + userId + ":" + assessmentId;  
  
    redisTemplate.opsForValue().set(key, role, 15, TimeUnit.MINUTES);  
}
```

Role permissions cache:

```
public void cacheRolePermissions(String role, Set<String> permissions){  
  
    String key = "perm:role:" + role;  
  
    redisTemplate.opsForSet().add(key, permissions.toArray(new String[0]));  
  
    redisTemplate.expire(key, 30, TimeUnit.MINUTES);  
}
```

7. GRANULAR CACHE EVICTION

When admin updates role or permission we delete only specific keys.

Example role change:

```
redisTemplate.delete("perm:role:AUDITOR");
```

Example user role update:

```
redisTemplate.delete("role:user:T100:U123:A101");
```

The next request will repopulate the cache.

8. MEMORY BASED EVICTION POLICY

Redis configuration:

```
maxmemory 4gb  
maxmemory-policy allkeys-lfu
```

Meaning:

Redis removes least frequently used keys when memory is full.

9. IMPORTANT THINGS TO FOCUS

-
- Use userId(UUID) instead of email for Redis keys
 - Avoid storing roles inside JWT
 - Always invalidate cache after role updates
 - Use Redis SET for permission lookup
 - Use cache-aside pattern for loading RBAC data
 - Never use global FLUSHALL in production
 - Ensure Redis TTL prevents stale permissions

=====

FINAL AUTHORIZATION FLOW

User Request
↓
JWT validated
↓
Extract tenantId + userId
↓
Extract assessmentId from API
↓
Redis lookup → role:user:{tenantId}:{userId}:{assessmentId}
↓
Role obtained
↓
Redis lookup → perm:role:{roleName}
↓
Permission list obtained
↓
Aspect checks required permission
↓
Allow request OR throw AccessDeniedException

=====

END

@Async

=====

@Async — History Service Summary

=====

1) Purpose in History Service

We use @Async in History Service to:

- Decouple audit/history persistence from main business flow
- Reduce API response time
- Prevent history failure from breaking main request
- Achieve eventual consistency

History is treated as non-blocking, secondary operation.

2) Basic Implementation

@EnableAsync // required at configuration level at main class

```
@Service
public class HistoryService {

    @Async("historyExecutor")
    public CompletableFuture<Void> saveAsync(HistoryEvent event) {
        try {
            historyRepository.save(event);
        } catch (Exception ex) {
            log.error("History persistence failed for {}", event.getId(), ex);
        }
        return CompletableFuture.completedFuture(null);
    }
}
```

3) How It Works Internally

```
Caller Thread
  ↓
Spring AOP Proxy
  ↓
TaskExecutor.submit()
  ↓
Separate background thread executes saveAsync()
```

Key Points:

- Runs in different thread
- Separate transaction context

- Exceptions do NOT propagate to caller
- Main request returns immediately

4) Ensuring Main Request Does Not Fail

- Async method runs in separate thread
- Exceptions are caught inside method
- No exception bubbles back to controller
- History failure does not affect API response

5) Critical Data Consistency Requirement

History must run **ONLY** after main transaction commits.

Correct pattern:

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void handle(TaskUpdatedEvent event) {
    historyService.saveAsync(event);
}
```

This ensures:

- No history written for rolled-back transactions

6) Retry for Transient Failures

```
@Async("historyExecutor")
@Retryable(
    value = { Exception.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 2000)
)
public CompletableFuture<Void> saveAsync(HistoryEvent event) {
    historyRepository.save(event);
    return CompletableFuture.completedFuture(null);
}
```

Solves:

- Temporary DB failure
- Connection pool hiccups
- Deadlocks

7) Thread Pool Configuration

```
-----  
  
@Bean(name = "historyExecutor")  
public Executor historyExecutor() {  
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();  
    executor.setCorePoolSize(5);  
    executor.setMaxPoolSize(10);  
    executor.setQueueCapacity(500);  
    executor.initialize();  
    return executor;  
}
```

Prevents:

- Blocking request threads
- Silent task dropping
- Thread exhaustion

8) Limitations of @Async for History

- In-memory execution (lost if app crashes)
- No durability guarantee
- No built-in retry unless added
- No delivery tracking
- No exactly-once guarantee

9) When @Async Is Acceptable for History

- Audit is non-critical
- Minor data loss acceptable
- Low-to-moderate traffic
- Eventual consistency allowed

10) When @Async Is NOT Enough

If history is:

- Compliance-critical
- Financially regulated
- Requires guaranteed persistence
- Needs high throughput

Better approach:

- Outbox Pattern
- Kafka-based event-driven architecture

11) Interview-Ready Summary

In our system, History Service was executed asynchronously to prevent audit logging from increasing API latency. The async method runs in a dedicated thread pool and is triggered only after the main transaction commits using `@TransactionalEventListener(AFTER_COMMIT)`. Failures are logged and retried when necessary. Since `@Async` is thread-based and not crash-safe, stronger durability guarantees would require an event-driven approach such as Kafka or the Outbox pattern.

=====

END

=====

StatusCodeExceptionHandling

=====

Exception Handling Using @ResponseStatus (Handling Specific HTTP Status Codes)

=====

1) Purpose

@ResponseStatus is used to map a specific exception to a specific HTTP status code.

It allows:

- Clean separation of business logic and HTTP response logic
- Automatic HTTP status mapping
- Avoiding manual ResponseEntity creation everywhere

2) Basic Usage on Custom Exception

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class HistoryNotFoundException extends RuntimeException {
    public HistoryNotFoundException(String message) {
        super(message);
    }
}
```

When this exception is thrown:

```
throw new HistoryNotFoundException("History not found");
```

Spring automatically returns:

HTTP 404 Not Found

3) With Custom Reason

```
@ResponseStatus(
    value = HttpStatus.BAD_REQUEST,
    reason = "Invalid history request"
)
public class InvalidHistoryRequestException extends RuntimeException {
}
```

Response:

HTTP 400 Bad Request

Body:

"Invalid history request"

Note:

Using "reason" produces default error response body.

4) Using @ExceptionHandler (More Control)

For structured JSON responses, prefer global exception handling:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(HistoryNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleHistoryNotFound(
        HistoryNotFoundException ex) {

        return new ErrorResponse(
            404,
            ex.getMessage(),
            LocalDateTime.now()
        );
    }
}
```

This gives:

- Custom JSON body
- Control over structure
- Consistent API contract

5) Handling Multiple Status Codes

Different exceptions → different status codes:

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class HistoryNotFoundException extends RuntimeException {}
```

```
@ResponseStatus(HttpStatus.BAD_REQUEST)
public class InvalidHistoryRequestException extends RuntimeException {}
```

```
@ResponseStatus(HttpStatus.CONFLICT)
public class DuplicateHistoryException extends RuntimeException {}
```

6) When Exception Is Thrown in Service

```
@Service
public class HistoryService {

    public History getHistory(Long id) {
        return historyRepository.findById(id)
            .orElseThrow(() ->
                new HistoryNotFoundException("History not found"));
    }
}
```

Spring automatically converts exception → HTTP status.

7) Difference: @ResponseStatus vs ResponseEntity

@ResponseStatus:

- Static mapping
- Cleaner
- Less boilerplate
- Good for simple APIs

ResponseEntity:

- Dynamic status
- Full response control
- Used when status depends on runtime condition

8) Limitations of @ResponseStatus

- Fixed HTTP status
- Harder to return dynamic error body
- Not ideal for complex error responses
- Does not handle validation errors automatically

9) Best Practice in Production

For structured APIs:

Use:

- @RestControllerAdvice
- @ExceptionHandler
- Custom error response object

Avoid using "reason" in @ResponseStatus for REST APIs

because it limits response body customization.

10) Interview-Ready Summary

@ResponseStatus allows mapping custom exceptions to specific HTTP status codes declaratively. When the exception is thrown, Spring automatically converts it to the defined HTTP status. For more structured error handling and custom JSON responses, it is recommended to use @RestControllerAdvice with @ExceptionHandler instead of relying solely on @ResponseStatus.

=====
END
=====

===== How Exceptions Are Thrown & Detected in Spring Boot =====

1) Do We Need to Explicitly Throw Exceptions?

Yes — for custom business exceptions, you must explicitly throw them.

Example:

```
if (history == null) {  
    throw new HistoryNotFoundException("History not found");  
}
```

Spring does NOT automatically throw your custom exceptions.
You define when business logic is violated.

2) Where Can Exceptions Be Thrown?

Exceptions can originate from:

- Controller layer
- Service layer
- Repository layer
- Validation layer
- Framework itself (Spring, Hibernate, Jackson, etc.)

Spring detects any unhandled exception that propagates up to the DispatcherServlet.

3) How Spring Detects Exceptions

Flow:

Controller → Service → Repository



Exception thrown



Bubbles up call stack



Spring DispatcherServlet catches it



Checks:

- @ExceptionHandler
- @RestControllerAdvice
- @ResponseStatus
- Default Exception Resolver



Converts to HTTP response

4) Example: Service Throws Exception

```
@Service
public class HistoryService {

    public History getHistory(Long id) {
        return historyRepository.findById(id)
            .orElseThrow(() ->
                new HistoryNotFoundException("History not found"));
    }
}
```

If not caught manually,
Spring automatically routes it to exception handlers.

5) Framework-Thrown Exceptions (Automatic)

Some exceptions are thrown automatically by Spring:

- `MethodArgumentNotValidException` (validation failure)
- `HttpMessageNotReadableException` (invalid JSON)
- `DataIntegrityViolationException` (DB constraint violation)
- `MissingServletRequestParameterException`

You do NOT throw these manually.
Spring throws them internally.

6) Can Spring Detect Exceptions From Anywhere?

Spring only handles exceptions that:

- Occur inside Spring-managed beans
- Happen during HTTP request processing
- Propagate to the DispatcherServlet

It will NOT detect:

- Exceptions caught and swallowed manually
- Exceptions inside separate threads unless handled properly
- Exceptions outside request lifecycle

7) What Happens If You Catch It?

```
try {  
    historyRepository.save(entity);  
} catch (Exception e) {  
    log.error("Failed");  
}
```

If you catch and do NOT rethrow,
Spring never sees it.

To let Spring handle it:

```
catch (Exception e) {  
    throw new HistorySaveException("Failed to save history");  
}
```

8) Async Case (Important)

For @Async methods:

- Exception does NOT propagate to controller thread.
- Must handle inside async method.
- Or use CompletableFuture and handle via exceptionally().

9) Summary

- Business exceptions must be explicitly thrown.
- Spring automatically catches uncaught exceptions.
- Exceptions bubble up the call stack.

- @ExceptionHandler / @ResponseStatus decide HTTP response.
- If you catch and suppress exception, Spring cannot detect it.
- Async exceptions do not propagate to request thread.

```
=====
END
=====
```


Deployment Strategy

Rolling Update Deployment Strategy

1) Purpose

Rolling Update is a deployment strategy where:

- New version instances are deployed gradually
- Old version instances are terminated progressively
- System remains available during deployment
- No full downtime occurs

Used in:

- Microservices
- Kubernetes-based deployments
- High-availability systems

2) How It Works

Instead of replacing all instances at once:

Old Version (v1):

Instance 1
Instance 2
Instance 3
Instance 4

Step-by-step process:

Step 1:

Terminate Instance 1 (v1)
Start Instance 1 (v2)

Step 2:

Terminate Instance 2 (v1)
Start Instance 2 (v2)

Continue until all are upgraded.

Traffic always flows to healthy instances.

3) Example in Kubernetes

spec:

replicas: 4

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

Meaning:

- At most 1 pod can be unavailable
- At most 1 extra pod can be created temporarily

4) Key Parameters

- maxUnavailable
Number of old pods that can go down during update.
- maxSurge
Extra pods allowed above desired replica count.
- Readiness Probe
Ensures traffic only goes to healthy pods.
- Liveness Probe
Restarts unhealthy pods.

5) Traffic Flow During Deployment

Load Balancer



Healthy Pods Only



Old + New versions coexist temporarily

Once new pod is Ready:

- It starts receiving traffic
- Old pod is removed

6) Advantages

- Zero downtime
- No traffic interruption
- Safe gradual rollout
- No duplicate environments required
- Resource-efficient

7) Limitations

- Old and new versions coexist → must be backward compatible
- DB schema changes must be handled carefully
- Slower rollout compared to recreate strategy

8) Data Consistency Considerations

When doing rolling update:

- ✓ Use backward-compatible APIs
- ✓ Avoid breaking DB changes
- ✓ Use feature flags if needed
- ✓ Apply DB migrations before deployment

Bad Example:

- New code expects new DB column
- Old pods still running → may break

9) Comparison With Other Strategies

Rolling Update:

- Gradual replacement
- No downtime
- Mixed versions temporarily

Blue-Green:

- Two separate environments
- Instant switch
- Higher infrastructure cost

Canary:

- Small % traffic to new version
- Risk-controlled rollout

10) Failure Handling

If new pod fails readiness:

- Kubernetes stops rollout
- Old pods continue serving traffic
- Rollback can be triggered

Deployment does NOT take down the entire system.

11) When To Use

- ✓ Stateless services
- ✓ High-availability APIs
- ✓ Microservices
- ✓ Kubernetes environments

Avoid if:

- ✗ Breaking schema changes
- ✗ Major incompatible changes

12) Interview-Ready Summary

Rolling Update is a zero-downtime deployment strategy where instances are gradually replaced with newer versions while keeping the service available. Traffic is routed only to healthy pods using readiness checks. It requires backward compatibility since old and new versions coexist temporarily. Kubernetes manages rollout using parameters like `maxUnavailable` and `maxSurge`, ensuring controlled and safe deployment.

=====

END

=====

Response code

=====

Important HTTP Response Codes & Meaning

=====

Status Code	Response Message	How to Set in ResponseEntity	When to Use
200	OK	<code>ResponseEntity.ok(body)</code>	Successful GET / PUT (with response body)
201	Created	<code>ResponseEntity.status(HttpStatus.CREATED).body(body)</code>	Resource created (POST)
202	Accepted	<code>ResponseEntity.status(HttpStatus.ACCEPTED).body(msg)</code>	Async processing (Kafka, @Async)
204	No Content	<code>ResponseEntity.noContent().build()</code>	Success but no response body (DELETE)
400	Bad Request	<code>ResponseEntity.badRequest().body(error)</code>	Invalid input / validation failure
401	Unauthorized	<code>ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(msg)</code>	Missing/invalid authentication (JWT issue)
403	Forbidden	<code>ResponseEntity.status(HttpStatus.FORBIDDEN).body(msg)</code>	Authenticated but no permission (RBAC fail)
404	Not Found	<code>ResponseEntity.status(HttpStatus.NOT_FOUND).body(msg)</code>	Resource not found

405	Method Not Allowed	<code>ResponseEntity.status(HttpStatus.METHOD_NOT_ALLOWED).build()</code>	Wrong HTTP method
408	Request Timeout	<code>ResponseEntity.status(HttpStatus.REQUEST_TIMEOUT).body(msg)</code>	Client request timeout
409	Conflict	<code>ResponseEntity.status(HttpStatus.CONFLICT).body(msg)</code>	Duplicate data / business conflict
410	Gone	<code>ResponseEntity.status(HttpStatus.GONE).body(msg)</code>	Resource permanently deleted
415	Unsupported Media Type	<code>ResponseEntity.status(HttpStatus.UNSUPPORTED_MEDIA_TYPE).body(msg)</code>	Wrong content type (not JSON/XML)
422	Unprocessable Entity	<code>ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(error)</code>	Validation errors (better than 400 in some APIs)
429	Too Many Requests	<code>ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS).body(msg)</code>	Rate limiting (API Gateway / IP limit)
500	Internal Server Error	<code>ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(msg)</code>	Generic server failure
501	Not Implemented	<code>ResponseEntity.status(HttpStatus.NOT_IMPLEMENTED).body(msg)</code>	Feature not supported
502	Bad Gateway	<code>ResponseEntity.status(HttpStatus.BAD_GATEWAY).body(msg)</code>	Downstream service failure
503	Service Unavailable	<code>ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).body(msg)</code>	Service down / maintenance
504	Gateway Timeout	<code>ResponseEntity.status(HttpStatus.GATEWAY_TIMEOUT).body(msg)</code>	Downstream timeout

Most Important Codes for Interviews

Must Remember:

200 → Success
201 → Created
204 → No Content
400 → Bad Request
401 → Unauthorized
403 → Forbidden
404 → Not Found
409 → Conflict
500 → Internal Server Error
503 → Service Unavailable

Difference Between Commonly Confused Codes

401 vs 403

401 → Not authenticated
403 → Authenticated but not authorized

400 vs 422

400 → Malformed request
422 → Validation/business rule failure

404 vs 403

404 → Resource doesn't exist
403 → Exists but no access

How Spring Sets Response Codes

Using `@ResponseStatus`:

`@ResponseStatus(HttpStatus.NOT_FOUND)`

Using ResponseEntity:

```
return ResponseEntity.status(404).body(...);
```

Using ErrorHandler:

```
@ExceptionHandler(...)
@ResponseStatus(HttpStatus.BAD_REQUEST)
```

=====

Interview-Ready Summary

=====

HTTP response codes are standardized 3-digit numbers that indicate the outcome of an HTTP request. They are grouped into 1xx informational, 2xx success, 3xx redirection, 4xx client errors, and 5xx server errors. Proper use of status codes improves API clarity, debugging, and contract correctness in distributed systems.

=====

END

=====

Service Boundry